

Self-Concordance, Cones, and CVXPY

Why the log barrier works, and how CVXPY exploits it

Dr. Robert Hildebrand

Spring 2026

Grado Department of Industrial & Systems Engineering
Virginia Tech

Where we left off

Lecture 1. We built an interior-point method from first principles:

- Log barrier $-\mu \sum \log(-g_i)$ smooths the KKT combinatorics.
- Central path $\mathbf{x}^*(\mu)$ satisfies a perturbed KKT system.
- Primal-dual Newton drives $\mu \downarrow 0$; IPOPT layers filter + inertia + restoration on top for nonconvex.

Two questions we kicked down the road:

1. *Why* does the log barrier work? What property of $-\sum \log x_i$ is actually being used?
2. How do we know the same trick works for **other constraint structures** — PSD matrices, second-order cones, exponentials?

Today's answer: **self-concordance** is the property that matters, and *every* convex cone admits a self-concordant barrier. That's what makes CVXPY work.

Today's roadmap

1. **Self-concordance** — the one inequality that makes Newton + barriers provably polynomial.
2. **The cone zoo** — LP, SOCP, SDP, exp, power — each with its barrier and complexity parameter ν .
3. **Coding break #1** — CVXPY in practice (LASSO, Markowitz, duals, SDP taste).
4. **Disciplined Convex Programming (DCP)** — the grammar of convex expressions. Why convexity is undecidable in general and how DCP routes around that.
5. **Coding break #2** — DCP as a proof system; ask CVXPY *why* an expression is or isn't convex.
6. **Conic lifting** — how CVXPY rewrites your problem into a cone program the solver can actually run.

Roadmap

Self-Concordance: The One Inequality

The Cone Zoo

Coding Break #1: CVXPY in Practice

Disciplined Convex Programming: The Grammar

Coding Break #2: DCP as a Proof System

Conic Lifting: From DCP to a Cone Program

Wrap-Up

What could go wrong with Newton on a barrier?

Newton's method converges quadratically *in a neighborhood* of the solution.

Two things we need to nail down:

1. **Size of that neighborhood.** Is it 10^{-3} ? 10^{-10} ? Does it shrink as $\mu \rightarrow 0$?
2. **Step length we can take.** Full steps can overshoot the barrier's domain; we need a principled damping rule.

The classical story (Boyd–Vandenberghe, Ch. 9): if the Hessian $\nabla^2 F$ is Lipschitz and $\nabla^2 F \succeq mI$, then Newton converges quadratically in a ball of size $\sim m/L$. But L and m depend on coordinates, units, problem scaling — not intrinsic.

Nesterov's fix (1988). Replace “Lipschitz Hessian” with a *self-referential* inequality: the third derivative of F is bounded by the second derivative *to the 3/2 power*. The resulting bounds are **affine-invariant** — independent of coordinates.

Self-concordance: the definition

A convex C^3 function $F : \text{int}(K) \rightarrow \mathbb{R}$ is **self-concordant** if

$$|F'''(\mathbf{x})[\mathbf{h}, \mathbf{h}, \mathbf{h}]| \leq 2 (F''(\mathbf{x})[\mathbf{h}, \mathbf{h}])^{3/2} \quad \forall \mathbf{x} \in \text{int}(K), \forall \mathbf{h}.$$

Reading it. The 1D directional third derivative in direction $\mathbf{h}$ is bounded by the $3/2$ power of the second directional derivative in the same direction.

Why the $3/2$? It's the unique exponent that makes the inequality *invariant under affine reparameterization*: $\mathbf{x} \mapsto A\mathbf{x} + \mathbf{b}$ preserves the bound. Any other exponent would break under coordinate changes.

One-dimensional examples:

- $F(x) = -\log x$ on $(0, \infty)$: $F''(x) = 1/x^2$, $F'''(x) = -2/x^3$. Check: $2/x^3 = 2(1/x^2)^{3/2}$. $\checkmark$
- $F(x) = x^2$: $F''(x) = 2$, $F'''(x) = 0$. $0 \leq 2 \cdot 2^{3/2}$. $\checkmark$ (every quadratic is SC)
- $F(x) = x^4$: $F''(x) = 12x^2$, $F'''(x) = 24x$. Inequality $24|x| \leq 2(12x^2)^{3/2}$ fails near $x = 0$. **Not SC.**

The log is the "canonical" SC function. Most useful SC barriers are built by summing or composing with logs and affine maps.

Self-concordant barrier: one more inequality

A self-concordant function F is a **self-concordant barrier** for K if additionally there exists $\nu \geq 1$ such that

$$\boxed{|F'(\mathbf{x})[\mathbf{h}]|^2 \leq \nu \cdot F''(\mathbf{x})[\mathbf{h}, \mathbf{h}]} \quad \forall \mathbf{x} \in \text{int}(K), \forall \mathbf{h}.$$

The smallest such ν is the **barrier parameter** (or **complexity parameter**) of F .

Equivalent formulation. Writing $\mathbf{h}^* = -(F''(\mathbf{x}))^{-1}F'(\mathbf{x})$ (the Newton direction for the barrier), the squared Newton decrement

$$\lambda_F(\mathbf{x})^2 = F'(\mathbf{x})^\top (F''(\mathbf{x}))^{-1} F'(\mathbf{x}) \leq \nu.$$

Reading it. ν upper-bounds how “big” the gradient of F can be relative to its Hessian. Small ν = the barrier is “tight” in a sense we’ll make quantitative.

The barrier parameter drives complexity. We’ll see that the number of IPM iterations to reduce μ by a factor $1/\varepsilon$ is

$$K = O(\sqrt{\nu} \log(1/\varepsilon)).$$

Smaller $\nu \Rightarrow$ fewer iterations.

The Newton decrement: convergence certificate

For a self-concordant F , define the **Newton decrement** at $\mathbf{x}$ as

$$\lambda_F(\mathbf{x}) = \sqrt{F'(\mathbf{x})^\top (F''(\mathbf{x}))^{-1} F'(\mathbf{x})}.$$

Properties:

- Affine-invariant (unlike $\|\nabla F\|$).
- $\lambda_F(\mathbf{x}) = 0 \iff \mathbf{x}$ is the minimizer of F .
- $\lambda_F(\mathbf{x})$ upper-bounds the *optimality gap*: $F(\mathbf{x}) - F(\mathbf{x}^*) \leq \omega(\lambda_F(\mathbf{x}))$ for $\omega(t) = t - \log(1 + t)$.

The two-phase convergence theorem [Nesterov 1988]. For the damped Newton step $\mathbf{x}^+ = \mathbf{x} - \alpha(F''(\mathbf{x}))^{-1}F'(\mathbf{x})$ with $\alpha = 1/(1 + \lambda_F(\mathbf{x}))$:

- **Damped phase** ($\lambda_F \geq 1/4$): each step reduces F by at least a constant, $\gamma = 0.09$.
- **Quadratic phase** ($\lambda_F < 1/4$): $\lambda_F(\mathbf{x}^+) \leq 2\lambda_F(\mathbf{x})^2$ — pure Newton with full step.

Solver pseudocode: `while  $\lambda_F > \varepsilon$ : if  $\lambda_F > 1/4$  take damped step, else take full step.`

The master complexity theorem

Theorem [Nesterov–Nemirovski 1994]. Let K be a solid convex cone with self-concordant barrier F of parameter ν . The path-following IPM that tracks $\mathbf{x}^*(\mu) = \arg \min_{\mathbf{x} \in \text{int}(K)} \mathbf{c}^\top \mathbf{x} + \mu F(\mathbf{x})$ as $\mu \downarrow 0$ converges in

$$K = O(\sqrt{\nu} \log(\nu/\varepsilon))$$

iterations to an ε -approximate solution.

What's remarkable.

- The bound depends on the barrier, not on the specific problem.
- Polynomial in the description size of K .
- Independent of the objective $\mathbf{c}$.

Consequence: every convex optimization is polynomial-time. Because every convex cone admits a self-concordant barrier (universal barrier theorem), every convex problem is solvable in polynomial time — a radical generalization of Karmarkar's 1984 LP result.

The key takeaway. Total complexity is

$$\underbrace{O(\sqrt{\nu} \log(1/\varepsilon))}_{\text{outer iterations}} \times \underbrace{O(\text{cost of one Newton system})}_{\text{per-iteration cost}}.$$

Smaller $\nu \Rightarrow$ fewer outer iterations. This is why **SOCP is often faster than an equivalent LP**: $\nu = 2$ per cone vs. $\nu = n$ for the orthant.

Roadmap

Self-Concordance: The One Inequality

The Cone Zoo

Coding Break #1: CVXPY in Practice

Disciplined Convex Programming: The Grammar

Coding Break #2: DCP as a Proof System

Conic Lifting: From DCP to a Cone Program

Wrap-Up

The cone zoo: five cones you'll meet in CVXPY

Every modern convex solver works on a Cartesian product of these cones.

Cone	Definition	ν
Zero cone $\{0\}^m$	— for equality constraints	0
Nonneg orthant $\mathbb{R}_+^n$	$\{x : x_i \geq 0\}$	n
Second-order cone $\mathcal{Q}^n$	$\{(t, \mathbf{x}) : t \geq \ \mathbf{x}\ _2\}$	2
PSD cone $\mathbb{S}_+^n$	$\{X = X^\top : X \succeq 0\}$	n
Exponential cone K_{exp}	$\text{cl}\{(x, y, z) : ye^{x/y} \leq z, y > 0\}$	3
Power cone K_α	$\{(x, y, z) : z \leq x^\alpha y^{1-\alpha}, x, y \geq 0\}$	3

Composition rule. Cartesian products of SC barriers are SC barriers with ν 's summing:

$$F_{K_1 \times K_2}(\mathbf{x}_1, \mathbf{x}_2) = F_1(\mathbf{x}_1) + F_2(\mathbf{x}_2), \quad \nu_{K_1 \times K_2} = \nu_1 + \nu_2.$$

So a problem mixing 10 SOC constraints and a 50×50 PSD gets $\nu = 10 \cdot 2 + 50 = 70$.

Solver support today (from CVXPY's perspective):

- ECOS: LP, SOC, exp. CLARABEL: LP, SOC, exp, power, SDP.

Barrier #1: the nonnegative orthant

Cone. $\mathbb{R}_+^n = \{\mathbf{x} \in \mathbb{R}^n : x_i \geq 0 \ \forall i\}.$

Barrier. $F(\mathbf{x}) = -\sum_{i=1}^n \log x_i.$

Derivatives. $\nabla F(\mathbf{x}) = (-1/x_i)_i$, $\nabla^2 F(\mathbf{x}) = \text{diag}(1/x_i^2)$, diagonal.

Verify self-concordance. Separates into 1D: for each i , $f_i(x) = -\log x$. We computed $|f'''| = 2/x^3 = 2(f'')^{3/2}$. ✓

Verify the ν -bound. The Newton decrement squared is

$$\lambda_F(\mathbf{x})^2 = \sum_{i=1}^n \frac{(1/x_i)^2}{1/x_i^2} = \sum_{i=1}^n 1 = n.$$

So $\nu = n$ is tight. (Maximized when $\mathbf{h}$ points in direction $(x_1, \dots, x_n)$.)

Consequence. LP with n non-negativity constraints: $K = O(\sqrt{n} \log(1/\varepsilon))$. This is the $O(\sqrt{n})$ Renegar-Gonzaga path-following bound.

This barrier is what IPOPT uses internally on the slack variables $\mathbf{s}^g$ (after it reformulates $g_i(\mathbf{x}) \leq 0$ as $g_i(\mathbf{x}) + s_i^g = 0, s_i^g \geq 0$).

Barrier #2: the second-order cone

Cone. $\mathcal{Q}^n = \{(t, \mathbf{x}) \in \mathbb{R} \times \mathbb{R}^{n-1} : t \geq \|\mathbf{x}\|_2\}$. An ice-cream cone.

Barrier. $F(t, \mathbf{x}) = -\log(t^2 - \|\mathbf{x}\|_2^2)$.

Let $\phi(t, \mathbf{x}) = t^2 - \|\mathbf{x}\|_2^2$ (concave on $\mathcal{Q}^n$). $F = -\log \phi$.

Gradient. $\nabla F = -(1/\phi) \nabla \phi = -\frac{1}{\phi} \begin{pmatrix} 2t \\ -2\mathbf{x} \end{pmatrix}$.

Hessian (after computation): $\nabla^2 F = \frac{4}{\phi} \begin{pmatrix} 1 & 0 \\ 0 & -I \end{pmatrix} + \frac{1}{\phi^2} (\nabla \phi)(\nabla \phi)^\top$. Not diagonal. But $\succ 0$ on $\text{int}(\mathcal{Q}^n)$.

Verify $\nu = 2$. After a short calculation (details: Nesterov–Todd 1997),

$$\lambda_F(t, \mathbf{x})^2 = \nabla F^\top (\nabla^2 F)^{-1} \nabla F \leq 2.$$

Why this matters: dimension doesn't affect complexity. Regardless of how big n is, one SOC contributes only $\nu = 2$ to the master theorem. So SOCP with k cones of *any* size: $\nu_{\text{total}} = 2k$, and $K = O(\sqrt{k} \log(1/\varepsilon))$ outer iterations.

Practical consequence. Reformulating a big LP as an SOCP (say, by representing the cost $\|\mathbf{c}\|_1$ with an SOC) can *reduce* iteration count. Rule of thumb: pick the cone that matches your constraint geometry, not the one that's linear.

Barrier #3: the positive semidefinite cone

Cone. $\mathbb{S}_+^n = \{X \in \mathbb{R}^{n \times n} : X = X^\top, X \succeq 0\}$ — dimension $n(n+1)/2$.

Barrier. $F(X) = -\log \det X$.

Why it works. $\log \det$ is concave on $\mathbb{S}_+^n$ (Minkowski's inequality). $\det X \rightarrow 0$ as X hits the boundary (an eigenvalue $\rightarrow 0$), so $F \rightarrow +\infty$.

Derivatives (matrix calculus):

$$\nabla F(X) = -X^{-1}, \quad \nabla^2 F(X)[H, H] = \text{tr}(X^{-1} H X^{-1} H).$$

Newton decrement. $\lambda_F(X)^2 = \text{tr}(X^{-1} X^{-1}) = \dots = n$ (for the gradient direction). So $\nu = n$.

Eigenvalue intuition. If $X = Q \Lambda Q^\top$ with $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$:

$$F(X) = -\sum_{i=1}^n \log \lambda_i.$$

The SDP barrier is the LP barrier on eigenvalues. Same $\nu = n$, same $O(\sqrt{n})$ complexity per PSD block.

A 50×50 SDP contributes $\nu = 50$. Its dimension (variables) is $50 \cdot 51/2 = 1275$. IPM iterations scale with $\sqrt{50} \approx 7$, not $\sqrt{1275} \approx 36$ — SDPs converge remarkably fast in *iteration count*, though each iteration can be expensive.

Barriers #4 and #5: exp and power cones

Exponential cone.

$$K_{\text{exp}} = \text{cl}\{(x, y, z) \in \mathbb{R}^3 : y \exp(x/y) \leq z, y > 0\}.$$

Captures constraints involving log, exp, entropy, KL-divergence.

Barrier. Not as clean as the others. Serrano (2015) gave the explicit form:

$$F(x, y, z) = -\log(y \log(z/y) - x) - \log z - \log y, \quad \nu = 3.$$

Power cone.

$$K_{\alpha} = \{(x, y, z) \in \mathbb{R}^3 : |z| \leq x^{\alpha} y^{1-\alpha}, x, y \geq 0\}, \quad \alpha \in (0, 1).$$

Captures p -norm constraints and x^p for rational p .

Barrier.

$$F(x, y, z) = -\log(x^{2\alpha} y^{2(1-\alpha)} - z^2) - \log x - \log y, \quad \nu = 3.$$

Why these cones matter in CVXPY. Many “awkward” convex constraints — entropy, log-sum-exp, p -norms, geometric means — are *not* expressible in LP + SOC + SDP. Exp and power cones make them tractable. Solver ECOS handles LP + SOC + exp; CLARABEL adds power + SDP.

Where does a new barrier come from?

Design recipe (most tractable SC barriers follow it):

Step 1. Express the cone as $K = \{\mathbf{x} : \phi_i(\mathbf{x}) \geq 0\}$ for concave ϕ_i .

Step 2. Try $F = -\sum_i \log \phi_i$.

Step 3. Verify self-concordance via the inequality (often reduces to known identities for sum/composition/affine-restriction).

Worked examples:

- $\mathbb{R}_+^n$: $\phi_i(\mathbf{x}) = x_i$ (affine, hence concave). $F = -\sum \log x_i$. $\nu = n$.
- $\mathcal{Q}^n$: $\phi(t, \mathbf{x}) = t^2 - \|\mathbf{x}\|^2$ (concave). $F = -\log \phi$. $\nu = 2$.
- $\mathbb{S}_+^n$: $\phi(X) = \det X$ (log-concave, not concave — so use $-\log \det$ directly).

Universal existence. Nesterov–Nemirovski (1994, Thm. 2.5.1): every n -dimensional convex cone admits an SC barrier with $\nu = O(n)$. Their construction uses

$$F_{\text{univ}}(\mathbf{x}) = \log \text{vol}\{\mathbf{y} \in K^* : \langle \mathbf{y}, \mathbf{x} \rangle \leq 1\}.$$

This is a **theoretical existence result** — not computable in practice. All cones you'll meet in CVXPY have hand-designed, tractable barriers.

Roadmap

Self-Concordance: The One Inequality

The Cone Zoo

Coding Break #1: CVXPY in Practice

Disciplined Convex Programming: The Grammar

Coding Break #2: DCP as a Proof System

Conic Lifting: From DCP to a Cone Program

Wrap-Up

Coding break #1: CVXPY in practice

Notebook: 02_cvxpy_dcp.ipynb

What we'll run together:

1. **LASSO regression** — $\min \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_1$. Under the hood: SOC + LP cones.
2. **Markowitz portfolio** — mean-variance QP with a budget constraint and optional SOC risk bound.
3. **Extracting dual variables** — `constraint.dual_value`. The KKT multipliers we derived in Lecture 1, now one line of code.
4. **Swapping solver backends** — ECOS vs. CLARABEL vs. SCS on the same problem. Compare iteration counts and solve times.
5. **An SDP taste** — a 3×3 PSD variable in a Max-Cut relaxation. Notice how cleanly $X \succeq 0$ becomes an LMI.
6. **Scaling experiment** — how does iteration count grow as n grows? ($\sqrt{n}$, not n — the master theorem at work.)

Things to look for. Solver choice (`prob.solver_stats.solver_name`), iteration count (`.num_iters`), solve time. Contrast the iteration count across problem sizes — it should

CVXPY basics: a two-minute refresher

```
import cvxpy as cp
import numpy as np

n = 10
A = np.random.randn(20, n)
b = np.random.randn(20)
lam = 0.1

x = cp.Variable(n)
obj = cp.Minimize(0.5 * cp.sum_squares(A @ x - b) + lam * cp.norm1(x))
prob = cp.Problem(obj)
prob.solve()

print("x*:", x.value)
print("solver:", prob.solver_stats.solver_name)
print("iters:", prob.solver_stats.num_iters)
print("solve time:", prob.solver_stats.solve_time)
```

What happened internally:

1. `cp.sum_squares`: squared ℓ_2 norm — DCP attributes (convex, nonneg, nondecreasing in $|\cdot|$).
2. `cp.norm1`: ℓ_1 norm — convex, nonneg, nondecreasing in $|\cdot|$.
3. Composition rule passes $\Rightarrow$ problem is DCP-compliant.
4. `prob.solve()`: graph-implementation rewrite $\rightarrow$ standard conic form $\rightarrow$ ECOS/CLARABEL.

Dual recovery: if you add constraints = `[x >= 0]` and call `constraints[0].dual_value` post-solve, you get the Lagrange multiplier vector.

Roadmap

Self-Concordance: The One Inequality

The Cone Zoo

Coding Break #1: CVXPY in Practice

Disciplined Convex Programming: The Grammar

Coding Break #2: DCP as a Proof System

Conic Lifting: From DCP to a Cone Program

Wrap-Up

Why DCP? Convexity is undecidable in general

Problem. You write

$$f(\mathbf{x}) = \log(\|\mathbf{x}\|^2 + \exp(x_1) - \cos(x_2)).$$

Is f convex?

Undecidability. Deciding whether an arbitrary expression is convex is undecidable in general (Ahmadi 2009, for polynomials of degree ≥ 4). No algorithm can correctly classify every convex expression.

Consequence. CVXPY cannot just “check convexity”. It would either reject convex problems (conservative) or accept non-convex ones (unsafe).

DCP's way out. Don't decide convexity. *Construct* convexity.

- Restrict the grammar to a set of *atomic functions* with known curvature.
- Combine them only with rules that *preserve* convexity.
- A problem that parses in this restricted grammar is convex *by construction*.

Tradeoff. You give up some convex problems (e.g. $\sqrt{x^2 + 1}$ is convex but not DCP-expressible as written). You gain: a *proof* of convexity that comes for free with every CVXPY program that parses.

DCP attributes: every atom carries three labels

Every function in CVXPY's atom library has three **curvature attributes**.

Curvature:

- **constant** — doesn't depend on variables.
- **affine** — $ax + b$; both convex and concave.
- **convex** — $f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$.
- **concave** — the opposite.

Sign: positive, negative, zero, unknown.

Monotonicity (per argument): nondecreasing, nonincreasing, nonmonotonic.

Atom	Curvature	Sign	Monotonicity
<code>square(x)</code>	convex	positive	nonmonotonic
<code>abs(x)</code>	convex	positive	nonmonotonic
<code>exp(x)</code>	convex	positive	nondecreasing
<code>log(x)</code>	concave	unknown	nondecreasing
<code>norm2(x)</code>	convex	positive	nondecreasing in $ x $
<code>log_sum_exp(x)</code>	convex	unknown	nondecreasing
<code>sqrt(x)</code>	concave	positive	nondecreasing

The composition rule

For $f(\mathbf{x}) = h(g_1(\mathbf{x}), g_2(\mathbf{x}), \dots, g_k(\mathbf{x}))$, DCP concludes f is **convex** if:

h is convex, and for each i , one of:

- h is nondecreasing in arg i and g_i is convex,
- h is nonincreasing in arg i and g_i is concave,
- g_i is affine.

Concave case: swap “convex” and “concave” throughout.

Works: `log_sum_exp(A @ x + b)`

- `log_sum_exp` is convex, nondecreasing in each arg.
- `A @ x + b` is affine $\Rightarrow$ rule passes.

$\Rightarrow$ convex. $\checkmark$

Fails: `log(exp(x) - exp(y))`

- `log` is concave, nondecreasing. But we're composing with `exp(x) - exp(y)` — convex minus convex, curvature *unknown*.
- Concave rule needs concave inside for nondecreasing args; unknown fails.

$\Rightarrow$ “DCP violation: does not follow DCP rules”.

“Fails DCP” $\neq$ “is non-convex.” CVXPY is conservative: some convex expressions fail DCP because they can't be *verified* by the composition rule. Sometimes you can rewrite.

DCP-valid, DCP-invalid, and “true but not DCP”

DCP-valid (clearly convex, clearly parses):

- `cp.square(x - 3)`: square convex nonmonotonic; $x - 3$ affine.
- `cp.norm(A @ x - b, 2)`: SOC epigraph.
- `cp.log_sum_exp(W @ x)`: softmax log-partition function.

DCP-invalid (genuinely non-convex):

- `cp.square(x) - cp.square(y)`: difference of convex. Non-convex in general. Rejected.
- `x * y`: bilinear. Rejected (not convex).

“True but not DCP” (convex, but not as written):

- `cp.sqrt(x**2 + 1)`: convex (composition of convex nondecreasing `sqrt` with convex x^2+1). But DCP sees `sqrt` as *concave-nondecreasing*, which needs a concave argument. Composition rule fails.
- *Fix*: rewrite as `cp.norm(cp.vstack([x, 1]), 2)` — an SOC epigraph. Mathematically equivalent, DCP-valid.

DCP is a **proof system**, not a classifier. A rejection doesn't mean non-convex — it means CVXPY can't *verify* convexity via the rule. Half the art of CVXPY is knowing when and how to rewrite.

Roadmap

Self-Concordance: The One Inequality

The Cone Zoo

Coding Break #1: CVXPY in Practice

Disciplined Convex Programming: The Grammar

Coding Break #2: DCP as a Proof System

Conic Lifting: From DCP to a Cone Program

Wrap-Up

Coding break #2: ask CVXPY why

Notebook: 02_5_dcp_proof_system.ipynb

Goal. Stop treating DCP as a black box. Every CVXPY expression has introspection — we'll use it to watch the composition rule fire.

What we'll run:

1. **A clearly convex, clearly DCP expression** — inspect `expr.is_convex()`, `.curvature`, `.sign`, `.is_dcp()`.
2. **A “true but not DCP” example** — $\sqrt{x^2 + 1}$ is convex but CVXPY rejects. We'll see *why* via the attribute chain, and rewrite using `cp.norm`.
3. **Composition rule at work** — `exp(convex)` passes, `exp(concave)` fails. Watch the curvature propagate up the expression tree.
4. **Signs matter** — `pos_const * convex_expr` is convex; `neg_const * convex_expr` is concave. The sign attribute drives the monotonicity check.
5. **Difference-of-convex** — genuinely non-convex, rejected by DCP.
6. **Constraints: what's allowed** — `convex  $\leq$  concave`, `affine = affine`, nothing else.

Roadmap

Self-Concordance: The One Inequality

The Cone Zoo

Coding Break #1: CVXPY in Practice

Disciplined Convex Programming: The Grammar

Coding Break #2: DCP as a Proof System

Conic Lifting: From DCP to a Cone Program

Wrap-Up

The graph implementation: each atom knows its epigraph

Key idea. Every DCP atom f comes with a **graph implementation**: a representation of its epigraph

$$\text{epi}(f) = \{(\mathbf{x}, t) : f(\mathbf{x}) \leq t\}$$

as a set of cone constraints over fresh auxiliary variables. When CVXPY lifts your problem, each atom is replaced by its graph.

Minimal example: $\|\mathbf{x}\|_2$. The epigraph is

$$\{(\mathbf{x}, t) : \|\mathbf{x}\|_2 \leq t\} = \{(\mathbf{x}, t) : (t, \mathbf{x}) \in \mathcal{Q}^{n+1}\}.$$

So `cp.norm(x, 2)` gets replaced by an auxiliary t and the constraint $(t, \mathbf{x})$ in SOC. The objective (or upper-bounding constraint) then sees t instead of `cp.norm(x, 2)`.

ℓ_1 norm. $\|\mathbf{x}\|_1 = \sum_i |x_i|$. Epigraph: $\sum_i u_i \leq t, -u_i \leq x_i \leq u_i$. Fresh vars $\mathbf{u}$; LP constraints only.

Max eigenvalue. $\lambda_{\max}(X)$ for symmetric X . Epigraph:
 $t \geq \lambda_{\max}(X) \iff tI - X \succeq 0$. One LMI.

– $\log \det X$. Requires a sequence of Schur complements plus n exp-cone constraints. Not pretty, but CVXPY handles it for you.

This is called “conic reformulation” or “graph lifting.” The lifted problem is equivalent, *is* a cone program in standard form, and can be solved by any conic IPM.

The full CVXPY pipeline, one figure

Input (your Python):

```
x = cp.Variable(n)
obj = cp.Minimize(cp.norm(A @ x - b, 2) + lam * cp.norm1(x))
prob = cp.Problem(obj, [x >= 0])
prob.solve()
```

Pipeline inside `prob.solve()`:

1. **Parse.** Build expression tree. Each node carries curvature + sign + monotonicity.
2. **DCP verification.** Walk the tree bottom-up via the composition rule. If any node's curvature is "unknown", raise `DCPError`.
3. **Graph rewrite.** Replace each atom by its epigraph / hypograph via auxiliary variables. E.g. `cp.norm(A @ x - b, 2)` becomes `t` with `(t, A@x-b)` in SOC.
4. **Canonicalize.** Flatten to standard conic form

$$\min \mathbf{c}^\top \mathbf{z} \quad \text{s.t.} \quad \mathbf{A}_{\text{eq}} \mathbf{z} = \mathbf{b}_{\text{eq}}, \mathbf{z} \in K_1 \times K_2 \times \cdots \times K_p.$$

5. **Solve.** Ship to a conic IPM (ECOS, CLARABEL, SCS, MOSEK) that runs the self-concordant-barrier machinery from today.
6. **Unpack.** Map the conic solution back to your original `x.value` and `constraint.dual_value`.

You can see the intermediate form via `prob.get_problem_data(solver="CLARABEL")`. Inspect `c`, `A`, `b`, and the cone list. It's often much bigger than your original — that's the price of auxiliary variables.

Example: what CVXPY does with a norm objective

You write:

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2$$

CVXPY lifts to:

$$\min_{\mathbf{x}, t} t \quad \text{s.t.} \quad (t, \mathbf{Ax} - \mathbf{b}) \in \mathcal{Q}^{m+1}.$$

A cone program with one SOC of size $m + 1$. CLARABEL runs an IPM on this.

You write:

$$\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \lambda \|\mathbf{x}\|_1$$

CVXPY lifts to:

$$\min_{\mathbf{x}, t, \mathbf{u}} t^2 + \lambda \mathbf{1}^\top \mathbf{u} \quad \text{s.t.} \quad (t, \mathbf{Ax} - \mathbf{b}) \in \mathcal{Q}^{m+1}, \quad -\mathbf{u} \leq \mathbf{x} \leq \mathbf{u}.$$

Wait — t^2 isn't a linear objective! So CVXPY introduces another auxiliary: let $s \geq t^2$, i.e. $(s, t) \in \textit{rotated SOC}$. Final:

$$\min_{\mathbf{x}, s, \mathbf{u}} s + \lambda \mathbf{1}^\top \mathbf{u} \quad \text{s.t.} \quad (s, \tfrac{1}{2}, \mathbf{Ax} - \mathbf{b}) \in \mathcal{Q}_{\text{rot}}^{m+2}, \quad -\mathbf{u} \leq \mathbf{x} \leq \mathbf{u}.$$

Conic form: $n + 1 + n = 2n + 1$ variables, one rotated SOC + LP constraints. ECOS/CLARABEL solves this in 10–20 iterations.

Moral. Your one-line CVXPY problem may blow up into a mid-sized cone program under the hood. This is normal and efficient — the solvers are designed for exactly this structure.

Putting it all together: Nesterov–Nemirovski + DCP + conic IPMs

The full chain from your expression to a solution:

1. You write a DCP-compliant optimization problem.
2. DCP verifies convexity *syntactically* via the composition rule.
3. Graph implementations lift each atom to its epigraph cone constraint.
4. CVXPY flattens to standard form: $\min \mathbf{c}^\top \mathbf{z}$ s.t. $\mathbf{A}\mathbf{z} = \mathbf{b}$, $\mathbf{z} \in K_1 \times \cdots \times K_p$.
5. Each cone has a self-concordant barrier with known ν .
6. The solver runs path-following IPM with the product barrier,
 $\nu_{\text{total}} = \sum \nu_i$.
7. Master theorem guarantees $O(\sqrt{\nu_{\text{total}}} \log(1/\varepsilon))$ iterations.

The big picture

Self-concordance is the mathematical foundation that makes CVXPY possible. Without it, the IPM complexity bound exists only for LP. With it, *any* convex cone — and therefore any DCP-compliant convex problem — is solvable in polynomial time.

One more piece of the ecosystem. For problems you solve *repeatedly* with different parameters (like MPC), CVXPY can **pre-compile** the conic reformulation and generate C code via CVXPYgen. See notebook `03_cvxpygen_parametric.ipynb` for deployment.

Roadmap

Self-Concordance: The One Inequality

The Cone Zoo

Coding Break #1: CVXPY in Practice

Disciplined Convex Programming: The Grammar

Coding Break #2: DCP as a Proof System

Conic Lifting: From DCP to a Cone Program

Wrap-Up

Today in three sentences

1. **Self-concordance** is the one inequality ($|F'''| \leq 2(F'')^{3/2}$) that makes Newton on a barrier provably polynomial, with complexity controlled by the barrier parameter ν .
2. **Every convex cone** admits a self-concordant barrier — LP ($\nu = n$), SOC ($\nu = 2$), SDP ($\nu = n$), exp/power ($\nu = 3$) — and Cartesian products compose. Modern solvers (ECOS, CLARABEL, SCS, MOSEK) exploit this.
3. **CVXPY** uses DCP to *construct* convexity syntactically, then lifts your problem to a cone program a self-concordant-barrier IPM can solve.

Complexity facts to remember

- Master theorem: $K = O(\sqrt{\nu} \log(1/\varepsilon))$ outer IPM iterations.
- LP: $\nu = n$ variables. SDP: $\nu = n$ matrix side. SOC: $\nu = 2$ per cone (dimension-free!).
- Short-step analysis gives $\sqrt{\nu}$; long-step / Mehrotra gives ν worst case but is typically much faster.

Further reading

- **Nesterov & Nemirovski**, *Interior-Point Polynomial Algorithms in Convex Programming* (SIAM 1994) — **the** book. Chapter 2 introduces self-concordance; Ch. 3 the master theorem.
- **Boyd & Vandenberghe**, *Convex Optimization* (2004), Ch. 11 — pedagogically clean 1-page statements of the key results.
- **Renegar**, *A Mathematical View of Interior-Point Methods* (SIAM 2001) — very short, very rigorous.
- **Grant, Boyd, Ye** (2006), “Disciplined Convex Programming” in *Global Optimization* (Springer) — the DCP founding paper.
- **CVXPY documentation**: <https://www.cvxpy.org>. The “Advanced” section covers graph implementations and the solver interface.
- **MOSEK Modeling Cookbook**:
<https://docs.mosek.com/modeling-cookbook/> — a catalogue of conic reformulations for every convex function you might want.

Companion notebooks (part1_solvers):

- 02_cvxpy_dcp.ipynb — core CVXPY usage (code break #1).